

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA TP.HCM
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



LUẬN VĂN TỐT NGHIỆP

PHÁT TRIỂN NỀN TẢNG NHÚNG ĐA CHỨC NĂNG DỰA TRÊN YOCTO PROJECT CHO CÁC ỨNG DỤNG TỰ ĐỘNG HÓA TRONG CÔNG NGHIỆP

HK252

Giảng viên hướng dẫn: TS.Lê Trọng Nhân

STT	Họ & Tên	MSSV	Lớp	Đánh giá
1	Đỗ Tuấn Anh	2210050	MT22KT01	100%
2	Dương Minh Hiếu	2210978	MT22KT01	100%
3	Nguyễn Hạo Duy	2210512	MT22KT01	100%

TP.Hồ Chí Minh, tháng 5 năm 2025



Thông tin hội đồng chấm khóa luận tốt nghiệp

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số 476/QĐĐHCNTT ngày 30 tháng 02 năm 2024 của Hiệu trưởng Trường Đại học Bách Khoa - TP.HCM.

- | | |
|-------------|----------|
| 1. PGS. TS. | Chủ tịch |
| 2. TS. | Ủy viên |
| 3. ThS. | Thư ký |

Lời cảm ơn

Không ai đạt được điều gì đó to lớn mà không nhờ sự giúp đỡ của những người xung quanh, cho dù là trực tiếp hay gián tiếp đi nữa. Để hoàn thành được khóa luận này, tác giả may mắn nhận được nhiều sự giúp đỡ và hỗ trợ từ quý thầy, cô, anh chị, bạn bè và người thân. Tác giả xin dành những trang đầu tiên này để bày tỏ lòng tri ân của mình tới tất cả mọi người, những người đã đồng hành cùng nhóm trong khoảng thời gian vừa qua. Đầu tiên, tác giả xin gửi lời cảm ơn sâu sắc đến toàn thể các thầy cô của Trường Đại học Công nghệ Thông tin nói chung và các thầy cô khoa Mạng máy tính và Truyền thông nói riêng. Nhờ những kiến thức quý giá mà thầy cô đã truyền đạt, cũng như việc hỗ trợ tận tình trong suốt khoảng thời gian thực hiện, nhóm đã hoàn thành khóa luận và đạt được các kết quả đáng ghi nhận. Tác giả xin đặc biệt cảm ơn TS. Google là người đã truyền cảm hứng, tận tình hướng dẫn và hỗ trợ tận tình về kiến thức, tạo môi trường thuận lợi để nhóm có thể học hỏi, trao đổi với các bạn, các em trong nhóm nghiên cứu. Đây là những kiến thức, kinh nghiệm quý giá, không chỉ có tác dụng trong khóa luận tốt nghiệp này mà còn trong khoảng thời gian làm việc trong chặng đường tiếp theo. Cuối cùng, tôi xin bày tỏ lòng tri ân đến gia đình và người thân, những người đã luôn là những hậu phương vững chắc và luôn ủng hộ từng quyết định mà nhóm đưa ra. Mặc dù đã nỗ lực rất nhiều để luận văn được hoàn thiện nhất, song khó có thể tránh khỏi thiếu sót và hạn chế. Kính mong nhận được sự thông cảm và ý kiến đóng góp từ quý thầy cô và các bạn.

Mục Lục

1	Phần mở đầu	7
1.1	Đặt vấn đề	8
1.2	Lý do chọn đề tài	8
1.3	Mục tiêu nghiên cứu	8
1.4	Phạm vi và giới hạn nghiên cứu	8
1.5	Bố cục luận văn	8
2	Tổng quan dự án	9
3	Cơ sở lý thuyết	10
3.1	Giao tiếp phần cứng công nghiệp	11
3.2	Linux Device Driver	11
3.2.1	Kiến trúc Layer trong Yocto	11
3.2.2	Các thành phần chính trong Layer của Yocto Project	12
3.2.3	Recipe trong Yocto	15
3.2.4	BitBake Build Engine	16
3.2.5	Cross-compilation trong Yocto	19
3.3	Kiến trúc Linux Kernel và Kernel Module	21
3.3.1	Tổng quan Linux Kernel	21
3.3.2	User Space và Kernel Space	22
3.3.3	Linux Device Driver	22
3.3.4	Device Tree trong Linux Embedded	22
3.3.5	Kernel Module	22
3.3.6	Kernel Logging	23
3.3.7	Cơ chế build Kernel Module	23
3.3.8	Cơ chế tự động nạp module	23
3.4	Quy trình xây dựng hệ thống nhúng với Yocto	24
3.4.1	Quy trình build tổng quát	24



3.4.2	Tích hợp Layer tùy chỉnh	24
3.4.3	Build Pipeline trong Yocto	24
3.4.4	Tích hợp Driver vào Image	25
3.4.5	Runtime Verification	25
4	Thiết kế hệ thống	26
5	Triển khai và đánh giá	27
6	Sản phẩm demo tích hợp	28
7	Kết luận và hướng phát triển	29

Danh sách bảng

3.1	Các biến cơ bản trong Recipe	15
3.2	Các task chính của BitBake	17
3.3	Mô hình Cross-compilation	20
3.4	So sánh User Space và Kernel Space	22
3.5	Các loại Linux Device Driver	22
3.6	Các lệnh kiểm tra Kernel Module	25

Danh sách hình

Chương 1

Phần mở đầu

- 1.1 Đặt vấn đề
- 1.2 Lý do chọn đề tài
- 1.3 Mục tiêu nghiên cứu
- 1.4 Phạm vi và giới hạn nghiên cứu
- 1.5 Bố cục luận văn

1.1 Đặt vấn đề

1.2 Lý do chọn đề tài

1.3 Mục tiêu nghiên cứu

1.4 Phạm vi và giới hạn nghiên cứu

1.5 Bố cục luận văn

Chương 2

Tổng quan dự án

2.1 Tổng quan về tự động hóa công nghiệp

2.2 Tổng quan về hệ điều hành nhúng

2.3 Tổng quan về Yocto Project

2.4 Các công trình liên quan

Chương 3

Cơ sở lý thuyết

- 3.1** Giao tiếp phần cứng công nghiệp
- 3.2** Linux Device Driver
- 3.3** Bảo mật hệ điều hành Linux nhúng
- 3.4** Edge AI trong hệ thống nhúng
- 3.5** Giao diện người dùng trên thiết bị nhúng
- 3.6** Kết nối Cloud và IoT

3.1 Giao tiếp phần cứng công nghiệp

3.2 Linux Device Driver

3.2.1 Kiến trúc Layer trong Yocto

Yocto Project sử dụng kiến trúc phân lớp (Layer Architecture) nhằm tổ chức metadata và mã nguồn theo hướng module hóa, linh hoạt và dễ mở rộng. Đây là một trong những đặc điểm quan trọng giúp Yocto trở thành nền tảng xây dựng Linux Embedded có khả năng tùy biến cao và phù hợp với nhiều kiến trúc phần cứng khác nhau.

Trong các hệ thống Linux Embedded hiện đại, số lượng package, driver, thư viện và thành phần hệ thống thường rất lớn. Nếu toàn bộ metadata và source code được lưu tập trung trong một cấu trúc duy nhất, việc quản lý dependency, cập nhật phiên bản và bảo trì hệ thống sẽ trở nên phức tạp. Do đó, Yocto áp dụng mô hình phân lớp nhằm chia nhỏ hệ thống thành các thành phần độc lập, mỗi layer đảm nhiệm một nhóm chức năng riêng biệt.

Kiến trúc phân lớp cho phép tách biệt rõ ràng giữa các thành phần hệ thống như BSP (Board Support Package), middleware, application và các package tùy chỉnh. Nhờ đó, người phát triển có thể dễ dàng bổ sung hoặc loại bỏ một chức năng mà không ảnh hưởng đến các thành phần còn lại của hệ thống. Điều này đặc biệt quan trọng trong phát triển hệ thống nhúng, nơi mỗi nền tảng phần cứng có thể yêu cầu các driver và cấu hình riêng biệt.

Ngoài khả năng module hóa, Layer Architecture còn giúp tăng tính tái sử dụng của metadata và source code. Một layer có thể được sử dụng lại trong nhiều dự án khác nhau mà không cần chỉnh sửa đáng kể. Ví dụ, các layer hỗ trợ kiến trúc phần cứng, networking hoặc multimedia có thể được tích hợp vào nhiều hệ thống Linux Embedded khác nhau thông qua cơ chế add-layer của Yocto.

Bên cạnh đó, việc phân lớp còn hỗ trợ hiệu quả cho quá trình cộng tác và phát triển nhóm. Mỗi nhóm phát triển có thể quản lý một layer riêng biệt tương ứng với chức năng của mình mà không làm ảnh hưởng trực tiếp đến hệ thống tổng thể. Điều này giúp giảm xung đột mã nguồn và nâng cao khả năng bảo trì dự án trong các hệ thống quy mô lớn.

Một ưu điểm quan trọng khác của kiến trúc phân lớp là khả năng quản lý ưu tiên metadata thông qua cơ chế layer priority. Khi nhiều layer cùng chứa recipe hoặc cấu hình cho một package, BitBake sẽ xác định layer có mức ưu tiên cao hơn để sử dụng. Cơ chế này cho phép người phát triển tùy chỉnh hoặc ghi đè hành vi của package mà không cần chỉnh sửa trực tiếp source code gốc.

Ngoài ra, Layer Architecture còn giúp Yocto Project duy trì tính mở rộng và khả

năng nâng cấp hệ thống trong thời gian dài. Khi cần cập nhật phiên bản Linux Kernel, thêm driver mới hoặc tích hợp middleware khác, người phát triển chỉ cần bổ sung hoặc cập nhật layer tương ứng thay vì chỉnh sửa toàn bộ hệ thống build.

Nhờ các ưu điểm về tính module hóa, khả năng tái sử dụng, dễ bảo trì và hỗ trợ mở rộng hệ thống, kiến trúc phân lớp đã trở thành nền tảng cốt lõi trong thiết kế của Yocto Project và đóng vai trò quan trọng trong quá trình phát triển các hệ thống Linux Embedded hiện đại. Mỗi layer có thể chứa:

- Recipe (.bb)
- Configuration
- Patch
- Source code

3.2.2 Các thành phần chính trong Layer của Yocto Project

Trong Yocto Project, kiến trúc layer được xây dựng dựa trên cơ chế metadata nhằm tổ chức hệ thống build theo hướng module hóa và có khả năng mở rộng cao. Mỗi layer đóng vai trò như một đơn vị chức năng độc lập, có thể chứa đầy đủ các thành phần cần thiết để xây dựng phần mềm hoặc hệ thống Linux nhúng. Việc phân chia các thành phần trong layer giúp quá trình phát triển, bảo trì và tái sử dụng mã nguồn trở nên thuận tiện hơn. Các thành phần chính thường xuất hiện trong một layer bao gồm Recipe, Configuration, Patch và Source Code.

Recipe (.bb)

Recipe là thành phần cốt lõi trong Yocto Project, được sử dụng để mô tả toàn bộ quy trình xây dựng một package hoặc một thành phần phần mềm trong hệ thống nhúng. Các recipe thường có phần mở rộng **.bb** và được xử lý bởi BitBake Build Engine. Có thể xem recipe như một tập hợp metadata dùng để hướng dẫn hệ thống build thực hiện các công việc như tải mã nguồn, giải nén, biên dịch, cài đặt và đóng gói phần mềm.

Trong mỗi recipe, nhiều biến hệ thống được sử dụng để mô tả thông tin liên quan đến package. Các biến này bao gồm tên package, phiên bản, giấy phép sử dụng, dependency, vị trí source code và các task cần thực hiện trong quá trình build. Thông qua recipe, BitBake có thể tự động phân tích dependency giữa các package và xây dựng quy trình biên dịch phù hợp.

Ngoài vai trò quản lý build process, recipe còn hỗ trợ tính module hóa và tái sử dụng trong Yocto Project. Thay vì cấu hình thủ công từng bước biên dịch, người phát triển

chỉ cần mô tả metadata trong recipe để hệ thống tự động thực hiện toàn bộ pipeline build. Điều này giúp giảm đáng kể độ phức tạp trong quá trình phát triển hệ thống Linux nhúng quy mô lớn.

Recipe cũng hỗ trợ kế thừa thông qua cơ chế class inheritance. Nhờ đó, nhiều package có thể sử dụng chung một tập chức năng build mà không cần lặp lại mã cấu hình. Đây là một trong những đặc điểm quan trọng giúp Yocto Project trở thành nền tảng build có tính mở rộng và bảo trì cao.

Configuration

Configuration là thành phần chịu trách nhiệm quản lý và thiết lập môi trường build trong Yocto Project. Các file cấu hình thường được lưu trong thư mục `conf/` và chứa các biến hệ thống phục vụ quá trình xây dựng Linux Embedded. Đây là thành phần quan trọng giúp xác định cách thức hoạt động của toàn bộ hệ thống build.

Thông qua configuration, người phát triển có thể thiết lập kiến trúc phần cứng mục tiêu, compiler, toolchain, package manager, loại filesystem và nhiều tham số hệ thống khác. Ngoài ra, configuration còn cho phép quản lý danh sách layer được sử dụng trong môi trường build cũng như thứ tự ưu tiên giữa các layer.

Một trong những vai trò quan trọng của configuration là hỗ trợ khả năng tùy biến hệ thống Linux nhúng. Tùy thuộc vào yêu cầu phần cứng và ứng dụng, người phát triển có thể thay đổi các thông số cấu hình để tạo ra các image Linux tối ưu về kích thước, hiệu năng hoặc mức tiêu thụ tài nguyên.

Các file configuration trong Yocto thường được tổ chức theo nhiều cấp độ khác nhau như cấu hình toàn cục, cấu hình machine, cấu hình distro hoặc cấu hình riêng cho từng layer. Nhờ kiến trúc này, Yocto Project có khả năng quản lý các hệ thống build phức tạp với số lượng package và dependency rất lớn.

Ngoài chức năng quản lý môi trường build, configuration còn đóng vai trò quan trọng trong việc đảm bảo tính tái lập của hệ thống. Khi các thông số build được định nghĩa rõ ràng trong configuration, hệ thống có thể được build lại với kết quả tương đồng trên nhiều môi trường phát triển khác nhau.

Patch

Patch là thành phần được sử dụng để sửa đổi hoặc cập nhật mã nguồn trong quá trình build mà không cần thay đổi trực tiếp source code gốc. Trong Yocto Project, patch đóng vai trò quan trọng trong việc tùy biến phần mềm, sửa lỗi hệ thống và bổ sung chức năng mới cho các package.

Patch thường được lưu dưới dạng file `.patch` hoặc `.diff`. Các file này chứa thông tin thay đổi giữa phiên bản mã nguồn gốc và phiên bản đã được chỉnh sửa. Trong quá

trình build, BitBake sẽ tự động áp dụng patch trước khi thực hiện biên dịch mã nguồn.

Việc sử dụng patch mang lại nhiều lợi ích trong phát triển hệ thống nhúng. Trước hết, patch giúp duy trì tính nguyên vẹn của source code gốc, từ đó tạo thuận lợi cho việc cập nhật phiên bản mới hoặc đồng bộ với repository chính thức. Bên cạnh đó, patch còn hỗ trợ quản lý thay đổi một cách rõ ràng và có hệ thống, giúp quá trình bảo trì phần mềm trở nên dễ dàng hơn.

Trong các hệ thống Linux Embedded, patch thường được sử dụng để tối ưu kernel, sửa lỗi driver, tùy biến bootloader hoặc bổ sung hỗ trợ cho phần cứng mới. Đây là kỹ thuật phổ biến trong phát triển phần mềm nhúng vì nhiều thành phần hệ thống thường cần được tùy chỉnh để phù hợp với nền tảng phần cứng cụ thể.

Ngoài ra, patch còn đóng vai trò quan trọng trong việc quản lý vòng đời phần mềm. Thay vì chỉnh sửa trực tiếp source code, toàn bộ thay đổi được lưu trữ dưới dạng patch giúp dễ dàng theo dõi lịch sử chỉnh sửa, rollback hoặc áp dụng lại trên các phiên bản hệ thống khác nhau.

Source Code

Source Code là thành phần chứa mã nguồn của chương trình hoặc hệ thống phần mềm được sử dụng trong quá trình build. Đây là thành phần cốt lõi quyết định chức năng và hành vi của package trong hệ thống Linux Embedded.

Trong Yocto Project, source code có thể được lấy từ nhiều nguồn khác nhau như local file, Git repository, tarball package hoặc remote server. Hệ thống BitBake sẽ tự động tải và quản lý source code dựa trên thông tin được khai báo trong recipe.

Source code trong hệ thống nhúng thường bao gồm nhiều loại thành phần như application, library, kernel module, bootloader hoặc device driver. Tùy thuộc vào chức năng của package, source code có thể được viết bằng các ngôn ngữ như C, C++, Python hoặc Shell Script.

Một trong những đặc điểm quan trọng của source code trong Linux Embedded là khả năng tương tác trực tiếp với phần cứng hệ thống. Điều này đòi hỏi mã nguồn phải được tối ưu về hiệu năng, khả năng quản lý tài nguyên và độ ổn định trong quá trình vận hành.

Ngoài chức năng thực thi logic hệ thống, source code còn đóng vai trò quan trọng trong việc đảm bảo tính mở rộng và khả năng bảo trì phần mềm. Việc tổ chức mã nguồn theo kiến trúc module giúp hệ thống dễ dàng nâng cấp, tái sử dụng và tích hợp thêm các chức năng mới trong tương lai.

Trong các dự án Linux Embedded quy mô lớn, source code thường được quản lý thông qua hệ thống version control như Git nhằm hỗ trợ cộng tác nhóm, theo dõi lịch sử thay đổi và quản lý phiên bản phần mềm một cách hiệu quả.

3.2.3 Recipe trong Yocto

Trong Yocto Project, Recipe là thành phần trung tâm của hệ thống build và đóng vai trò mô tả toàn bộ quy trình xây dựng một package phần mềm. Recipe thường có phần mở rộng `.bb` (BitBake Recipe) và được BitBake sử dụng để thực hiện các tác vụ như tải mã nguồn, cấu hình môi trường build, biên dịch, cài đặt và đóng gói package.

Có thể xem recipe như một tập hợp metadata dùng để hướng dẫn BitBake cách xây dựng một thành phần phần mềm cụ thể trong hệ thống Linux Embedded. Thông qua recipe, Yocto Project có khả năng tự động hóa toàn bộ quy trình build và quản lý dependency giữa các package trong hệ thống.

Một recipe thường chứa nhiều biến và task nhằm mô tả thông tin liên quan đến package. Các biến này được sử dụng để khai báo metadata như tên package, phiên bản, license, vị trí source code và phương thức build. Trong khi đó, các task chịu trách nhiệm thực thi từng giai đoạn trong pipeline build của BitBake.

Một số biến quan trọng trong recipe được trình bày trong Bảng 3.1.

Biến	Ý nghĩa
SUMMARY	Mô tả package
LICENSE	Khai báo license
SRC_URI	Đường dẫn source code
S	Thư mục source
inherit	Kế thừa class

Bảng 3.1: Các biến cơ bản trong Recipe

Biến `SUMMARY` được sử dụng để mô tả ngắn gọn chức năng của package. Thông tin này giúp người phát triển dễ dàng nhận biết vai trò của package trong hệ thống build.

Biến `LICENSE` dùng để khai báo giấy phép sử dụng phần mềm. Việc quản lý license là yêu cầu quan trọng trong các hệ thống Linux Embedded nhằm đảm bảo tuân thủ các điều kiện sử dụng phần mềm mã nguồn mở.

Biến `SRC_URI` được sử dụng để xác định vị trí source code hoặc các file cần thiết cho quá trình build. Source code có thể được lấy từ nhiều nguồn khác nhau như local file, Git repository, tarball package hoặc remote server.

Biến `S` xác định thư mục chứa source code sau khi quá trình giải nén hoàn tất. Biến này giúp BitBake xác định chính xác vị trí thực hiện các task biên dịch và cài đặt.

Biến `inherit` cho phép recipe kế thừa các class build có sẵn trong Yocto Project. Cơ chế kế thừa giúp tái sử dụng các chức năng build phổ biến và giảm sự trùng lặp trong metadata.

Trong quá trình build, BitBake sẽ phân tích dependency giữa các recipe để xác định thứ tự thực thi phù hợp. Nhờ đó, hệ thống có thể tự động build toàn bộ Linux Embedded

Image với số lượng package lớn mà vẫn đảm bảo tính nhất quán.

Một đặc điểm quan trọng của recipe là khả năng tái sử dụng và mở rộng thông qua cơ chế append và class inheritance. Người phát triển có thể bổ sung hoặc ghi đè metadata của recipe mà không cần chỉnh sửa trực tiếp file gốc. Điều này giúp quá trình bảo trì hệ thống trở nên dễ dàng hơn, đặc biệt trong các dự án Linux Embedded quy mô lớn.

Ngoài ra, recipe còn đóng vai trò quan trọng trong việc đảm bảo tính tái lập của hệ thống build. Khi toàn bộ metadata và quy trình build được định nghĩa rõ ràng trong recipe, hệ thống có thể được build lại trên nhiều môi trường phát triển khác nhau với kết quả tương đồng.

Nhờ khả năng tự động hóa, quản lý dependency và hỗ trợ tùy biến mạnh mẽ, recipe đã trở thành thành phần cốt lõi trong kiến trúc của Yocto Project và là nền tảng quan trọng trong quá trình phát triển các hệ thống Linux Embedded hiện đại.

3.2.4 BitBake Build Engine

BitBake là build engine trung tâm của Yocto Project, chịu trách nhiệm phân tích metadata và thực thi các task được định nghĩa trong recipe nhằm xây dựng package hoặc toàn bộ hệ thống Linux Embedded. Có thể xem BitBake như một hệ thống tự động hóa build hoạt động dựa trên metadata, cho phép quản lý dependency, thực thi pipeline build và tạo ra image Linux hoàn chỉnh cho nền tảng nhúng.

Trong Yocto Project, mỗi recipe sẽ được BitBake phân tích để xác định các task cần thực hiện cũng như mối quan hệ phụ thuộc giữa các package. Dựa trên dependency graph, BitBake sẽ tự động xác định thứ tự thực thi phù hợp nhằm đảm bảo quá trình build diễn ra chính xác và nhất quán.

Một trong những ưu điểm quan trọng của BitBake là khả năng tự động hóa toàn bộ quá trình build. Thay vì thực hiện thủ công từng bước như tải source code, cấu hình compiler hoặc cài đặt package, BitBake cho phép toàn bộ pipeline build được mô tả thông qua metadata trong recipe. Điều này giúp giảm đáng kể độ phức tạp trong quá trình phát triển hệ thống Linux Embedded.

Trong quá trình build, BitBake sử dụng tập hợp các task để xử lý từng giai đoạn khác nhau. Mỗi task đại diện cho một bước cụ thể trong pipeline build và được thực thi theo trình tự xác định.

Các task chính của BitBake được trình bày trong Bảng 3.2.

Task	Chức năng
do_fetch	Tải source code
do_unpack	Giải nén source
do_patch	Áp dụng patch
do_compile	Biên dịch mã nguồn
do_install	Cài đặt package
do_package	Đóng gói package

Bảng 3.2: Các task chính của BitBake

Task do_fetch

Task `do_fetch` là giai đoạn đầu tiên trong pipeline build của BitBake. Nhiệm vụ chính của task này là tải source code hoặc các tài nguyên cần thiết phục vụ quá trình build package.

Source code có thể được lấy từ nhiều nguồn khác nhau như Git repository, HTTP server, FTP server, local file hoặc tarball package. BitBake sử dụng thông tin trong biến `SRC_URI` để xác định vị trí dữ liệu cần tải.

Trong quá trình fetch, BitBake cũng thực hiện kiểm tra checksum nhằm đảm bảo tính toàn vẹn của source code. Điều này giúp hạn chế lỗi build do source code bị thay đổi hoặc hư hỏng trong quá trình tải xuống.

Task `do_fetch` đóng vai trò quan trọng trong việc đảm bảo khả năng tái lập của hệ thống build. Khi source code được quản lý thông qua metadata, cùng một recipe có thể được sử dụng để build lại package trên nhiều môi trường khác nhau với kết quả tương đồng.

Task do_unpack

Sau khi source code được tải về, BitBake thực hiện task `do_unpack` nhằm giải nén và chuẩn bị mã nguồn cho các bước xử lý tiếp theo.

Trong giai đoạn này, các file nén như `.tar.gz`, `.zip` hoặc package source sẽ được giải nén vào thư mục làm việc của BitBake. Đối với source code được tải từ Git repository, task này sẽ thực hiện checkout source code vào workspace tương ứng.

Task `do_unpack` giúp chuẩn hóa cấu trúc source code trước khi chuyển sang các bước patch hoặc biên dịch. Điều này đảm bảo các task phía sau luôn làm việc trên một môi trường source code thống nhất.

Ngoài chức năng giải nén, task này còn hỗ trợ quản lý workspace build một cách tự động, giúp người phát triển không cần thao tác thủ công với source code trong quá trình build hệ thống.

Task do_patch

Task `do_patch` được sử dụng để áp dụng các patch lên source code sau khi quá trình giải nén hoàn tất. Đây là bước quan trọng trong việc tùy biến package hoặc sửa lỗi hệ thống mà không cần thay đổi trực tiếp mã nguồn gốc.

Patch thường được sử dụng trong các trường hợp:

- Sửa lỗi phần mềm.
- Tùy biến kernel hoặc driver.
- Bổ sung tính năng mới.
- Tối ưu hệ thống cho phần cứng cụ thể.

Trong quá trình thực thi task `do_patch`, BitBake sẽ tự động tìm và áp dụng các file patch được khai báo trong recipe. Nếu patch không thể áp dụng thành công, quá trình build sẽ dừng lại để tránh tạo ra package không nhất quán.

Việc sử dụng task patch giúp duy trì tính nguyên vẹn của source code gốc, đồng thời hỗ trợ quản lý thay đổi phần mềm một cách có hệ thống và dễ bảo trì hơn.

Task do_compile

Task `do_compile` chịu trách nhiệm biên dịch source code thành mã máy hoặc binary executable có thể sử dụng trên hệ thống đích.

Trong hệ thống Linux Embedded, quá trình compile thường sử dụng cross-compiler nhằm tạo binary phù hợp với kiến trúc phần cứng mục tiêu như ARM, x86 hoặc RISC-V. BitBake sẽ tự động thiết lập compiler, sysroot và môi trường build dựa trên metadata của hệ thống.

Task `do_compile` có thể thực hiện nhiều loại build khác nhau như:

- Build application.
- Build library.
- Build kernel module.
- Build Linux Kernel.

Trong quá trình compile, BitBake cũng quản lý dependency giữa các package nhằm đảm bảo các thư viện hoặc thành phần cần thiết đã được build trước đó.

Đây là một trong những task quan trọng nhất trong pipeline build vì nó quyết định trực tiếp đến khả năng hoạt động của package trên hệ thống đích.

Task do_install

Sau khi quá trình biên dịch hoàn tất, BitBake thực hiện task `do_install` để cài đặt các file output vào thư mục staging hoặc root filesystem tạm thời.

Trong giai đoạn này, các binary executable, library, configuration file và resource file sẽ được sắp xếp theo đúng cấu trúc filesystem chuẩn của Linux. Điều này giúp package có thể được tích hợp vào root filesystem của Embedded Linux Image.

Task `do_install` đóng vai trò quan trọng trong việc chuẩn hóa cấu trúc package trước khi đóng gói. Nếu quá trình install không đúng cấu trúc, package có thể không hoạt động chính xác trên hệ thống runtime.

Ngoài ra, task này còn hỗ trợ việc tách package thành nhiều thành phần khác nhau như runtime package, development package hoặc debug package nhằm tối ưu quá trình quản lý phần mềm trong Linux Embedded.

Task do_package

Task `do_package` là giai đoạn cuối trong pipeline build của BitBake, chịu trách nhiệm đóng gói các file đã được cài đặt thành package có thể triển khai trên hệ thống đích.

BitBake hỗ trợ nhiều định dạng package khác nhau như:

- RPM
- DEB
- IPK

Trong quá trình đóng gói, BitBake sẽ tự động phân tích dependency runtime giữa các package nhằm đảm bảo hệ thống có thể cài đặt và vận hành chính xác.

Task `do_package` giúp chuẩn hóa quá trình quản lý phần mềm trong Linux Embedded, đồng thời hỗ trợ cập nhật và bảo trì hệ thống một cách linh hoạt hơn.

Ngoài ra, các package được tạo ra từ task này còn có thể được tái sử dụng trong nhiều image khác nhau mà không cần build lại source code, góp phần tối ưu thời gian build và nâng cao hiệu quả phát triển hệ thống nhúng.

3.2.5 Cross-compilation trong Yocto

Trong các hệ thống Linux Embedded, kiến trúc phần cứng của thiết bị đích thường khác với kiến trúc của máy tính dùng để phát triển phần mềm. Do đó, quá trình biên dịch mã nguồn không thể thực hiện trực tiếp bằng compiler thông thường trên hệ thống

host. Để giải quyết vấn đề này, Yocto Project sử dụng cơ chế Cross-compilation nhằm hỗ trợ biên dịch phần mềm cho nhiều nền tảng phần cứng khác nhau.

Cross-compilation là quá trình biên dịch mã nguồn trên một hệ thống máy tính nhưng chương trình được tạo ra sẽ chạy trên một hệ thống khác có kiến trúc phần cứng khác biệt. Trong Linux Embedded, quá trình này thường diễn ra khi source code được build trên máy tính x86 nhưng binary output được thiết kế để chạy trên thiết bị ARM, RISC-V hoặc các nền tảng nhúng chuyên dụng khác.

Cơ chế cross-compilation đóng vai trò rất quan trọng trong phát triển hệ thống nhúng vì phần lớn thiết bị embedded có tài nguyên hạn chế, không đủ khả năng thực hiện quá trình biên dịch trực tiếp trên thiết bị. Việc build trên máy host giúp tận dụng hiệu năng xử lý cao của máy tính phát triển, từ đó giảm đáng kể thời gian build và tối ưu quy trình phát triển phần mềm.

Trong mô hình cross-compilation, hệ thống thường bao gồm ba thành phần chính: Build Machine, Host Machine và Target Machine. Mỗi thành phần đảm nhiệm một vai trò khác nhau trong quá trình xây dựng phần mềm.

Các thành phần trong mô hình cross-compilation được trình bày trong Bảng 3.3.

Thành phần	Vai trò
Build Machine	Máy thực hiện build
Host Machine	Máy chạy toolchain
Target Machine	Thiết bị nhúng đích

Bảng 3.3: Mô hình Cross-compilation

Build Machine là hệ thống thực hiện toàn bộ quá trình build package hoặc image Linux Embedded. Đây thường là máy tính cá nhân hoặc server phát triển có hiệu năng xử lý cao và chạy hệ điều hành Linux. Build Machine chịu trách nhiệm thực thi BitBake, quản lý metadata và điều phối toàn bộ pipeline build trong Yocto Project.

Host Machine là hệ thống chứa và thực thi toolchain dùng cho quá trình cross-compilation. Trong nhiều trường hợp, Build Machine và Host Machine có thể là cùng một thiết bị vật lý. Tuy nhiên, về mặt kiến trúc logic, Host Machine đại diện cho môi trường chạy compiler và các công cụ build cần thiết để tạo binary cho hệ thống đích.

Target Machine là thiết bị nhúng đích nơi chương trình sau khi biên dịch sẽ được triển khai và thực thi. Đây có thể là board ARM, gateway công nghiệp, hệ thống IoT hoặc các nền tảng embedded chuyên dụng khác. Kiến trúc phần cứng của Target Machine thường khác với kiến trúc của Build Machine, do đó cần sử dụng cross-compiler phù hợp để tạo binary tương thích.

Trong Yocto Project, cross-compilation được quản lý tự động thông qua hệ thống metadata và toolchain. BitBake sẽ tự động lựa chọn compiler, linker và thư viện phù

hợp với kiến trúc của target device dựa trên cấu hình machine và distro. Điều này giúp giảm đáng kể độ phức tạp trong quá trình phát triển Linux Embedded.

Một thành phần quan trọng trong cross-compilation là cross-toolchain. Toolchain là tập hợp các công cụ phục vụ quá trình build như compiler, assembler, linker và debugger. Trong hệ thống embedded, toolchain thường được thiết kế riêng cho từng kiến trúc phần cứng nhằm đảm bảo binary output có thể hoạt động chính xác trên thiết bị đích.

Ngoài compiler, Yocto còn sử dụng sysroot để hỗ trợ cross-compilation. Sysroot là môi trường filesystem chứa các header file, library và dependency cần thiết cho quá trình build. Việc sử dụng sysroot giúp compiler truy cập đúng thư viện và tránh xung đột giữa môi trường host và target.

Một ưu điểm quan trọng của cross-compilation trong Yocto là khả năng tự động hóa và tái lập hệ thống build. Khi toàn bộ toolchain và dependency được quản lý thông qua metadata, hệ thống có thể được build lại trên nhiều môi trường khác nhau mà vẫn đảm bảo tính nhất quán của binary output.

Ngoài ra, cơ chế cross-compilation còn giúp tối ưu hiệu năng phát triển phần mềm nhúng. Thay vì biên dịch trực tiếp trên thiết bị embedded có tài nguyên hạn chế, quá trình build được thực hiện trên máy tính hiệu năng cao, từ đó rút ngắn thời gian build và tăng hiệu quả phát triển hệ thống.

Nhờ khả năng hỗ trợ đa kiến trúc phần cứng, tự động quản lý toolchain và tối ưu quy trình build, cross-compilation đã trở thành thành phần cốt lõi trong Yocto Project và đóng vai trò quan trọng trong quá trình phát triển các hệ thống Linux Embedded hiện đại.

3.3 Kiến trúc Linux Kernel và Kernel Module

3.3.1 Tổng quan Linux Kernel

Linux Kernel là thành phần lõi của hệ điều hành Linux, chịu trách nhiệm quản lý tài nguyên hệ thống.

Các chức năng chính của kernel:

- Quản lý tiến trình.
- Quản lý bộ nhớ.
- Quản lý thiết bị.
- Quản lý hệ thống tập tin.

3.3.2 User Space và Kernel Space

Linux chia hệ thống thành hai không gian:

User Space	Kernel Space
Ứng dụng người dùng	Driver và Kernel
Quyền hạn thấp	Quyền hạn cao

Bảng 3.4: So sánh User Space và Kernel Space

Quá trình truy cập phần cứng:

Application



System Call



Kernel



Driver



Hardware

3.3.3 Linux Device Driver

Device Driver là thành phần trung gian giữa hệ điều hành và phần cứng.

Các loại driver phổ biến:

Loại Driver	Ví dụ
Character Device	UART
Block Device	SSD
Network Device	Ethernet

Bảng 3.5: Các loại Linux Device Driver

Driver RS485 thuộc nhóm Character Device Driver.

3.3.4 Device Tree trong Linux Embedded

3.3.5 Kernel Module

Kernel Module là thành phần có thể nạp hoặc gỡ khỏi kernel trong thời gian chạy mà không cần biên dịch lại toàn bộ kernel.

Các lệnh quản lý module:

insmod
rmmod
modprobe
lsmod

Cấu trúc cơ bản của kernel module:

module_init()
module_exit()

Trong đó:

- module_init() được gọi khi nạp module.
- module_exit() được gọi khi gỡ module.

3.3.6 Kernel Logging

Linux Kernel sử dụng hàm `printk()` để ghi log hệ thống.

Ví dụ:

```
printk("RS485 Driver Loaded");
```

Thông điệp kernel có thể kiểm tra bằng lệnh:

```
dmesg
```

3.3.7 Cơ chế build Kernel Module

Kernel module được build thông qua kernel build system.

Ví dụ Makefile:

```
obj-m := rs485_driver.o
```

Dòng lệnh trên khai báo module cần build.

```
$(MAKE) -C $(KERNEL_SRC) M=$(SRC) modules
```

Cho phép sử dụng kernel source tree để biên dịch external module.

3.3.8 Cơ chế tự động nạp module

Yocto hỗ trợ tự động nạp kernel module khi hệ thống khởi động bằng biến:

```
KERNEL_MODULE_AUTOLOAD += "rs485_driver"
```


3.4 Quy trình xây dựng hệ thống nhúng với Yocto

3.4.1 Quy trình build tổng quát

Quy trình xây dựng hệ thống nhúng bằng Yocto:

Source Code

↓

Recipe (.bb)

↓

BitBake

↓

Cross Compilation

↓

Kernel Module (.ko)

↓

Root Filesystem

↓

Embedded Linux Image

3.4.2 Tích hợp Layer tùy chỉnh

Layer được thêm vào môi trường build bằng lệnh:

```
bitbake-layers add-layer ../meta-rs485-driver
```

Việc tích hợp layer giúp BitBake nhận diện recipe và metadata của driver.

3.4.3 Build Pipeline trong Yocto

Pipeline build gồm:

Fetch

→ Configure

→ Compile

→ Install

→ Package

→ RootFS

→ Image

3.4.4 Tích hợp Driver vào Image

Driver được tích hợp vào image bằng:

```
IMAGE_INSTALL:append = " rs485-driver"
```

Khi build image, driver sẽ được đưa vào root filesystem.

3.4.5 Runtime Verification

Sau khi boot hệ thống, module được kiểm tra bằng các lệnh:

Lệnh	Chức năng
lsmod	Kiểm tra module đang nạp
dmesg	Kiểm tra kernel log
modinfo	Xem thông tin module

Bảng 3.6: Các lệnh kiểm tra Kernel Module

Chương 4

Thiết kế hệ thống

Chương này tập trung vào thiết kế 3 thành phần kỹ thuật cốt lõi của nền tảng: Driver phần cứng, Bảo mật hệ thống và Công cụ hỗ trợ phát triển.

- 4.1** Kiến trúc tổng thể
- 4.2** Thiết kế Device Driver
- 4.3** Thiết kế Bảo mật Hệ thống
- 4.4** Thiết kế Configuration Tool

Chương 5

Triển khai và đánh giá

Chương này trình bày quá trình cài đặt, tích hợp và kết quả kiểm thử từng thành phần kỹ thuật của nền tảng.

5.1 Môi trường phát triển

5.2 Hiện thực Device Driver

5.3 Hiện thực Bảo mật Hệ thống

5.4 Hiện thực Configuration Tool

Chương 6

Sản phẩm demo tích hợp

Chương này là minh chứng thực nghiệm cho toàn bộ nền tảng: một hệ thống tự động hóa công nghiệp hoàn chỉnh được xây dựng trên nền tảng đã phát triển ở các chương trước.

- 6.1** Tổng quan sản phẩm demo
- 6.2** Kiến trúc hệ thống demo
- 6.3** Thiết kế và triển khai từng module
- 6.4** Tích hợp và kiểm thử sản phẩm demo

Chương 7

Kết luận và hướng phát triển

7.1 Kết luận

7.2 Hướng phát triển